

Design and Analysis of Computer Algorithms HW2

Solution Version

Course: Design and Analysis of Computer Algorithms

Exam: Homework 2

Source: HW2.pdf

Year: 2026

1. Prove that exponential functions a^n have different orders of growth for different values of base $a > 0$. (Hint: Consider two arbitrary different bases a_1 and a_2 .)

Reference Answer

Let a_1 and a_2 be two positive bases with $a_1 < a_2$. Then

$$\frac{a_1^n}{a_2^n} = \left(\frac{a_1}{a_2}\right)^n.$$

Since $0 < \frac{a_1}{a_2} < 1$, we have

$$\lim_{n \rightarrow \infty} \left(\frac{a_1}{a_2}\right)^n = 0.$$

Therefore $a_1^n = o(a_2^n)$. In particular, $a_1^n = O(a_2^n)$, but $a_2^n \neq O(a_1^n)$ because

$$\frac{a_2^n}{a_1^n} = \left(\frac{a_2}{a_1}\right)^n \rightarrow \infty.$$

Thus different positive bases give different asymptotic orders of growth.

2. Please derive the growth order of the following sum:

$$\sum_{i=0}^{n-1} \sum_{j=0}^{i-1} (i+j).$$

Use the $\Theta(g(n))$ notation with the simplest function $g(n)$ possible.

Reference Answer

First compute the inner sum:

$$\sum_{j=0}^{i-1} (i+j) = \sum_{j=0}^{i-1} i + \sum_{j=0}^{i-1} j = i^2 + \frac{i(i-1)}{2} = \frac{3i^2 - i}{2}.$$

Hence

$$\sum_{i=0}^{n-1} \sum_{j=0}^{i-1} (i+j) = \sum_{i=0}^{n-1} \frac{3i^2 - i}{2} = \frac{3}{2} \sum_{i=0}^{n-1} i^2 - \frac{1}{2} \sum_{i=0}^{n-1} i.$$

Since $\sum_{i=0}^{n-1} i^2 = \Theta(n^3)$ and $\sum_{i=0}^{n-1} i = \Theta(n^2)$, the cubic term dominates. Therefore,

$$\sum_{i=0}^{n-1} \sum_{j=0}^{i-1} (i+j) = \Theta(n^3).$$

3. Give the tightest asymptotic upper bounds for $T(n)$ in each of the following recurrences. Please provide the reason for your answer.

- (a) $T(n) = 10T(\frac{n}{3}) + 5n^2$
- (b) $T(n) = T(\lfloor \frac{n}{2} \rfloor) + 2$
- (c) $T(n) = T(\sqrt{n}) + \Theta(\lg \lg n)$
- (d) $T(n) = T(\frac{1}{5}n) + T(\frac{4}{5}n) + \Theta(n)$

Reference Answer

(a) By the Master Theorem, $a = 10$, $b = 3$, and $f(n) = 5n^2 = \Theta(n^2)$. Since

$$n^{\log_b a} = n^{\log_3 10}$$

and $2 < \log_3 10$, we have $f(n) = O(n^{\log_3 10 - \epsilon})$ for some $\epsilon > 0$. Therefore,

$$T(n) = \Theta(n^{\log_3 10}),$$

so the tightest asymptotic upper bound is $O(n^{\log_3 10})$.

(b) Expanding the recurrence gives

$$T(n) = T\left(\left\lfloor \frac{n}{2^k} \right\rfloor\right) + 2k.$$

Strictly, repeated flooring is not exactly the same as $\lfloor \frac{n}{2^k} \rfloor$, but floors can change the subproblem size only by a constant factor at this scale, so they do not affect the asymptotic number of levels. The recursion reaches a constant-size input after $k = \Theta(\lg n)$ levels. Hence

$$T(n) = \Theta(\lg n),$$

so the tightest asymptotic upper bound is $O(\lg n)$.

(c) Let $m = \lg n$ and define $S(m) = T(2^m)$. Since $\sqrt{n} = 2^{\frac{m}{2}}$, the recurrence becomes

$$S(m) = S\left(\frac{m}{2}\right) + \Theta(\lg m).$$

At level k , the nonrecursive cost is proportional to

$$\lg\left(\frac{m}{2^k}\right) = \lg m - k.$$

The recursion stops when $\frac{m}{2^k}$ becomes constant. Let $L = \lfloor \lg m \rfloor$. Then

$$\begin{aligned} S(m) &= \Theta\left(\sum_{k=0}^L (\lg m - k)\right) \\ &= \Theta\left((L+1) \lg m - \frac{L(L+1)}{2}\right) \\ &= \Theta((\lg m)^2). \end{aligned}$$

Substituting back $m = \lg n$ gives

$$T(n) = \Theta((\lg \lg n)^2),$$

so the tightest asymptotic upper bound is $O((\lg \lg n)^2)$.

(d) Use the Akra-Bazzi theorem. We find p from

$$\left(\frac{1}{5}\right)^p + \left(\frac{4}{5}\right)^p = 1.$$

The solution is $p = 1$. With $g(n) = \Theta(n)$,

$$T(n) = \Theta\left(n^p \left(1 + \int_1^n \frac{g(u)}{u^{p+1}} du\right)\right) = \Theta\left(n \left(1 + \int_1^n \frac{u}{u^2} du\right)\right) = \Theta(n \lg n).$$

Thus the tightest asymptotic upper bound is $O(n \lg n)$.

4. A set of points (finite or infinite) in the plane is called convex if for any two points P and Q in the set, the entire line segment with the endpoints at P and Q belongs to the set. The convex hull of a set S of points is the smallest convex set containing S in the plane. The convex-hull problem is the problem of constructing the convex hull for a given set S of n points. One of the solution to construct the convex-hull for a given set is to use the divide-and-conquer strategy. You can find such a solution from many Algorithm textbook or by googling it on WWW.

Now, we consider the shortest path around problem defined as follows. There is a fenced area in the two-dimensional Euclidean plane in the shape of a convex polygon with vertices at points $P_1(x_1, y_1), P_2(x_2, y_2), \dots, P_n(x_n, y_n)$ (not necessarily in this order). There are two more points $A(x_A, y_A)$ and $B(x_B, y_B)$, such that $x_A < \min\{x_1, x_2, \dots, x_n\}$ and $x_B > \max\{x_1, x_2, \dots, x_n\}$. Design a reasonably efficient algorithm for computing the length of the shortest path between A and B . Note that the path cannot cross inside the fenced area but it can go along the fence. Please argue the correctness and time complexity of your algorithm.

Reference Answer

Idea. Since the fence is a convex polygon, any shortest valid path from A to B either goes around the upper side of the polygon or around the lower side. It consists of a straight segment from A to a tangent vertex, a boundary chain along the polygon, and a straight segment from another tangent vertex to B .

A tangent vertex from an external point X to the convex hull is a hull vertex where a line through X touches the hull without crossing its interior. For each external point, there are two such vertices: one for the upper tangent and one for the lower tangent.

Shortest-Path-Around-Convex-Fence($P[1..n]$, A , B):

- 1 compute the convex hull H of $P[1..n]$ in clockwise order
- 2 find the two tangent vertices from A to H : A_u and A_l
- 3 find the two tangent vertices from B to H : B_u and B_l
- 4
- 5 compute $L_u \leftarrow \text{dist}(A, A_u) + \text{boundary-length}(A_u, B_u, \text{upper chain}) + \text{dist}(B_u, B)$
- 6 compute $L_l \leftarrow \text{dist}(A, A_l) + \text{boundary-length}(A_l, B_l, \text{lower chain}) + \text{dist}(B_l, B)$
- 7 **return** $\min(L_u, L_l)$

Here A_u, B_u are the tangent vertices on the upper side of the hull, and A_l, B_l are the tangent vertices on the lower side. The boundary lengths are computed by summing Euclidean distances of consecutive hull vertices along the corresponding chain.

Correctness. Consider any shortest valid path from A to B . If the path touches the fence, any portion of the path outside the fence between two fixed contact points must be a straight segment; otherwise we could replace it by a shorter straight segment. At the first and last contact points, the straight segments from A and B must be tangent to the convex polygon; if either segment entered the interior, the path would be invalid, and if it touched a non-tangent point, it could be shortened by moving the contact point along the boundary until a tangent is reached.

Once the two tangent contact points are fixed, the path cannot cross the interior of the convex polygon, so the boundary portion must follow one of the two polygon chains between them. Since A lies strictly to the left of the polygon and B lies strictly to the right, these are exactly the upper chain and the lower chain. The algorithm computes the length of both possible shortest forms and returns the smaller one, so it returns the shortest valid path length.

Time complexity. The convex hull can be computed in $O(n \lg n)$ time. After the hull is in cyclic order, the tangent vertices and the two chain lengths can be found by scanning the hull in $O(h)$ time, where $h \leq n$ is the number of hull vertices. Therefore the total running time is $O(n \lg n)$.

5. Recall the Maximum Subarray problem we discussed in class and answer the following questions.

- Argue that a maximum subarray of $A[1..j+1]$ is either a maximum subarray of $A[1..j]$ or a subarray $A[i..j+1]$, for some $1 \leq i \leq j+1$.
- Please use the above observation to develop a nonrecursive, linear-time algorithm for the maximum subarray problem.

Reference Answer

- Let M be a maximum subarray of $A[1..j+1]$.

If M does not contain the last element $A[j+1]$, then M lies entirely inside $A[1..j]$. Since M is maximum for the larger array, no subarray of $A[1..j]$ can have a larger sum, so M is also a maximum subarray of $A[1..j]$.

If M contains $A[j+1]$, then because a subarray is contiguous, it must have the form $A[i..j+1]$ for some $1 \leq i \leq j+1$.

Therefore a maximum subarray of $A[1..j+1]$ is either a maximum subarray of $A[1..j]$ or a suffix subarray ending at $j+1$.

- Maintain two values while scanning from left to right:

- ending: the maximum sum of a subarray that must end at the current position.
- best: the maximum subarray sum seen anywhere so far.

Linear-Maximum-Subarray($A[1..n]$):

```

1  let ending  $\leftarrow A[1]$ 
2  let best  $\leftarrow A[1]$ 
3  let ending-left  $\leftarrow 1$ 
4  let best-left  $\leftarrow 1$ 
5  let best-right  $\leftarrow 1$ 
6
7  for  $j \leftarrow 2$  to  $n$ :
8      if  $\text{ending} + A[j] < A[j]$ :
9          ending  $\leftarrow A[j]$ 
10         ending-left  $\leftarrow j$ 
11     else:
12         ending  $\leftarrow \text{ending} + A[j]$ 
13
14     if ending  $>$  best:
15         best  $\leftarrow$  ending
16         best-left  $\leftarrow$  ending-left
17         best-right  $\leftarrow j$ 
18
19  return (best-left, best-right, best)

```

The recurrence behind the scan is

$$\text{ending}[j] = \max(A[j], \text{ending}[j-1] + A[j]).$$

This exactly chooses the best suffix ending at j : either start a new subarray at j , or extend the best suffix ending at $j-1$. By part (a), the global optimum for $A[1..j]$ is either the old global optimum for $A[1..j-1]$ or a suffix ending at j , so updating best with ending is correct.

The loop performs constant work for each array element, so the running time is $\Theta(n)$ and the extra space usage is $\Theta(1)$.